

Propuesta de sintaxis para nuestro lenguaje de programación equipo 6

Luis Ernesto Hernandez Rosas

Uriel Balderas Aguilar

Alberto Sebastian Barradas Gonzalez

Reglas Fundamentales para asignación de símbolos:

1. Evitar el uso de caracteres de letras
2. Usar hasta 3 caracteres contiguos
3. Evitar el uso de caracteres no presentes en un teclado convencional
4. Respetar el uso de ciertos caracteres en su contexto (<= , >= , < , > , = son todos relacionales numéricos; +, -, *, /, % son todos operaciones)

Comentarios	
/* comentario */	comentario unilínea
/* comentario */	comentario multilínea

Tipos de Datos	
#	Integer
#.	Float
#..	Double
@	String
?’	Boolean
[]	Arreglo
[>]	Lista

Ejemplo de Asignación	
Tipo variable <- valor ;	
#. calificacion <- 10 ;	Se declara una variable “calificacion” con precisión de punto flotante y valor de 10.0 (aun si no tiene punto, al ser tipo flotante se asume)

Definimos la siguiente estructura para las funciones:

nombreFuncion (INPUT) -> OUTPUT {

```
...
:: retorno ;
}
```

Definimos la siguiente estructura para los condicionales:

Estructuras de Flujo	
Condicional (IF)	
Lenguaje	Equivalencia
c1 ? X : c2 ? Y : c3 ? Z : E ;	if(c1) then X else if(c2) then Y else if(c3) then Z else E
c1 ? X : c2 ? Y : c3 ? Z : E ;	if(c1) then X else if(c2) then Y else if(c3) then Z else E
Loop - Ciclo	
Lenguaje	Equivalencia
<< código >>	while(true) X
<< condición ? X : !! >>	while(condición) X
<< X ; ! condición ? !!; >>	do X while(condición)
inicio ; << condición ? X ; avance : !! ; >>	for(inicio ; condición ; avance) X
inicio1 ; <<	for(inicio1 ; condición1 ; avance1) for(inicio2 ; condición2 ; avance2)

condición1 ? inicio2 ; << condicion2 ? X ; avance2 : !! >> avance1 : !! ; >>	X
--	---

Palabras Reservadas	
Lenguaje	Equivalente
::	return (when used inside of a returning procedure)
!!	break
<!	continue
<-	asignación
?	if
:	else
:	else if
<<	begin loop
>>	end loop
{	begin (explicit) sequence
}	end (explicit) sequence
->	of Type (indicates expected function return type)
;	Separates instructions inside of a sequence

lexer detecta los tokens, es el parser quien decide
parser(mucha estructura ¿cuando escuchará el token?)
cuidado con las ambigüedades
bison avisa de ambigüedades